TERRAFORM — COMPLETE STUDY NOTES

Azure-focused | Interview Ready | Code + Explanation

PHASE 1 — THE MENTAL MODEL

What problem does Terraform solve

- **Before Terraform: ClickOps** — logging into cloud portals and manually clicking through menus to create infrastructure
- Problems with manual (ClickOps) approach:
  - Slow and error-prone
  - Cannot reproduce exactly
  - Cannot track what changed and when
  - Expensive and time-consuming to rebuild
- Terraform solves this by writing infrastructure as code
- Code can be tested, scanned, peer reviewed and placed under source control

What is Terraform

- A tool developed by HashiCorp
- Single binary command line tool — no dependent libraries
- Platform agnostic — works with Azure, AWS, GCP and many others
- Unlike AWS CloudFormation which only works with AWS
- HCL = HashiCorp Configuration Language — language Terraform files are written in
- JSON is also supported but only for machine-generated code, not humans

Declarative vs Imperative

- Terraform is DECLARATIVE
  - You declare WHAT you want (desired state)
  - Terraform figures out HOW to get there
  - You do not write step-by-step instructions
- Analogy:
  - Declarative → "I want a pizza with cheese and olives"
  - Imperative → "First make the dough, then spread sauce, then add cheese..."

⚠ NOTE: Because Terraform is declarative, if you change a value in code Terraform figures out how to go from the current state to the new state automatically.

Cattle vs Pets Mindset

- **Pets** — named, cared for, nursed when sick, expensive to maintain
- **Cattle** — numbered, replaceable, disposed when no longer needed
- Old school servers = Pets (hand built, kept alive forever)
- Cloud infrastructure with Terraform = Cattle (spin up, tear down, rebuild anytime)

- Terraform enables the cattle approach because infrastructure is defined in code

PHASE 2 — THE CORE 3

PROVIDERS

What is a Provider

- The bridge between Terraform and the cloud platform
- Terraform by itself knows nothing about Azure or AWS
- Provider is the plugin specific to a cloud service provider
- Downloads automatically when you run terraform init
- Multiple providers can be configured in the same project

Provider Block Syntax (Azure)

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "azurerm" {
  features {}
}
```

Line by Line Explanation

`terraform {}` → configures Terraform itself

`required_providers` → declares which providers this project needs

`azurerm` → the Azure Resource Manager provider name

`source = "hashicorp/azurerm"` → where to download it from (Terraform Registry)

`version = "~> 3.0"` → use version 3.x, do not jump to 4.x — version pinning

`provider "azurerm"` → block to configure the provider

`features {}` → required by Azure provider — can be empty but must exist

RESOURCES

What is a Resource

- The actual things you create on Azure — VMs, VNets, Resource Groups, Storage Accounts
- Every infrastructure component Terraform manages is a resource block
- Provider = connection to Azure | Resource = create THIS specific thing on Azure

Resource Block Syntax

```
resource "azurerm_resource_group" "my_rg" {
  name     = "myResourceGroup"
```

```
  location = "East US"
}
```

Line by Line Explanation

`resource` → keyword telling Terraform to create something

`"azurerm_resource_group"` → resource TYPE — what to create, always starts with provider name (azurerm)

`"my_rg"` → local name — what YOU call it inside Terraform code. Azure does not see this.

`name = "myResourceGroup"` → actual name that appears in Azure portal

`location = "East US"` → Azure region where resource will be created

Resource Reference Pattern (MEMORISE THIS)

```
Format:  resource_type.local_name.attribute
azurerm_resource_group.my_rg.name
azurerm_resource_group.my_rg.location
azurerm_virtual_network.my_vnet.id
azurerm_subnet.my_subnet.id
azurerm_linux_virtual_machine.my_vm.id
azurerm_public_ip.my_pip.ip_address
azurerm_network_interface.my_nic.private_ip_address
```

Resource Referencing Another Resource

```
resource "azurerm_virtual_network" "my_vnet" {
  name                = "myVNet"
  location            = azurerm_resource_group.my_rg.location
  resource_group_name = azurerm_resource_group.my_rg.name
  address_space       = ["10.0.0.0/16"]
}
```

`azurerm_resource_group.my_rg.location` → reading location from another resource instead of hardcoding

`azurerm_resource_group.my_rg.name` → reading the name from the resource group — creates implicit dependency

IMPLICIT vs EXPLICIT DEPENDENCIES

Implicit Dependency

- When one resource references another using interpolation
- Terraform automatically detects the dependency and creates the referenced resource first

```
resource "azurerm_subnet" "my_subnet" {
  name                 = "mySubnet"
  resource_group_name  = azurerm_resource_group.my_rg.name
  virtual_network_name = azurerm_virtual_network.my_vnet.name
  address_prefixes     = ["10.0.1.0/24"]
}
```

`azurerm_virtual_network.my_vnet.name` → Terraform sees this reference and creates VNet before creating Subnet

Explicit Dependency using depends_on

- Used when Terraform cannot detect the dependency automatically
- Example: VM bootstrap script pulls from a Storage Account but there is no reference between them in code

```
resource "azurerm_linux_virtual_machine" "my_vm" {
  name                = "myVM"
  location            = "East US"
  resource_group_name = azurerm_resource_group.my_rg.name
  size                = "Standard_B1s"
  admin_username      = "azureuser"

  depends_on = [
    azurerm_storage_account.my_storage
  ]
}
```

`depends_on = [...]` → explicitly declares a dependency — Terraform creates the storage account first

STATE

What is the State File

- A file called terraform.tfstate — Terraform's memory of what it has created
- Automatically generated and managed by Terraform — do NOT edit manually
- Stored as JSON format
- Maps your configuration code to the real-world infrastructure in Azure

Why State is Important

- Every time you run terraform plan, Terraform compares:
  - Your .tf code → vs → State file → shows the difference
- Decision logic:
  - Resource in code but not in state → CREATE it
  - Resource in state but not in code → DESTROY it
  - Resource in both but config changed → UPDATE it
  - Resource in both, no change → DO NOTHING

Remote Backend Configuration (Azure)

```
terraform {
  backend "azurerm" {
    resource_group_name  = "tfstate-rg"
    storage_account_name = "cognizanttfstate"
    container_name       = "tfstate"
    key                  = "prod/terraform.tfstate"
  }
}
```

`backend "azurerm"` → store state in Azure Blob Storage instead of locally

`resource_group_name` → the RG that holds the storage account

`storage_account_name` → the Azure Storage Account where state lives

`container_name` → the blob container inside that storage account

`key = "prod/terraform.tfstate"` → path and filename of the state file — using folders like prod/ organises multiple environments

State Locking

- Prevents two people from modifying state simultaneously
- Azure Blob Storage has locking BUILT IN using blob leasing — no extra setup needed

- AWS needs a separate DynamoDB table for locking
- When terraform apply starts:
  - 1. Terraform requests a lease on the state blob
  - 2. Azure grants the lease — state file is now locked
  - 3. Terraform makes its changes
  - 4. Terraform releases the lease
- If someone else tries to run terraform apply while locked:
  - Error: Error acquiring the state lock
  - Lock Info: ID: 12345678, Who: allen@cognizant.com

State Commands

```
terraform state list                                # list all tracked resources
terraform state show azurerm_resource_group.my_rg  # show details of one resource
terraform state rm azurerm_resource_group.my_rg    # remove from state (NOT from
Azure)
terraform force-unlock <LOCK_ID>                   # release a stuck lock — use
with caution
terraform show                                     # display full state in human-
readable format
```

PHASE 3 — HCL SYNTAX

VARIABLES (Input Variables)

What are Variables

- Make Terraform code flexible and reusable
- Instead of hardcoding values, define them once and use everywhere
- Can be overridden from outside: CLI, .tfvars file, environment variables
- Think of them like function parameters in programming

Variable Block Syntax

```
variable "location" {
  description = "The Azure region to deploy resources"
  type        = string
  default     = "East US"
}
```

`variable "location"` → declaring a variable named location

`description` → documents what this variable is for (optional but best practice)

`type = string` → this variable only accepts text values

`default = "East US"` → fallback value if no value is provided

Variable Types with Examples

```
variable "vm_count" {
  type    = number
  default = 2
}

variable "enable_monitoring" {
```

```
  type    = bool
  default = true
}

variable "allowed_ports" {
  type    = list(string)
  default = ["22", "80", "443"]
}

variable "tags" {
  type = map(string)
  default = {
    environment = "dev"
    team        = "devops"
  }
}
```

`string` → text value e.g. "East US"

`number` → numeric value e.g. 2 or 10

`bool` → true or false only

`list(string)` → ordered list of text values

`map(string)` → key-value pairs like a dictionary

Variable with Validation

```
variable "vm_size" {
  description = "The size of the VM"
  type        = string
  validation {
    condition     = contains(["Standard_B1s", "Standard_B2s"], var.vm_size)
    error_message = "VM size must be Standard_B1s or Standard_B2s"
  }
}
```

`validation {}` → block that enforces constraints on the variable value

`condition` → the check to perform

`error_message` → message shown if the condition fails

Variable Reference Pattern (MEMORISE THIS)

Format: `var.variable_name` **Example:** `var.location`

```
location = var.location
size     = var.vm_size
count    = var.vm_count
tags     = var.tags
```

Overriding Variables

- Command line:

```
terraform apply -var="location=West Europe"
```

- terraform.tfvars file (auto-loaded by Terraform):

```
location    = "West Europe"
environment = "prod"
```

- Environment variable:

```
export TF_VAR_location="West Europe"
```

- Priority order (highest to lowest):

- Command line → terraform.tfvars → environment variable → default

OUTPUTS

What are Outputs

- Capture and display values that are only known AFTER terraform apply
- Example: public IP of a VM, ID of a resource group
- Think of outputs as a receipt after a purchase
- Used for: displaying to terminal, sharing between modules, debugging

Output Block Syntax

```
output "resource_group_id" {
  description = "The ID of the resource group"
  value       = azurerm_resource_group.my_rg.id
}
```

`output "resource_group_id"` → declaring an output named resource_group_id

`description` → documents what this output represents

`value = azurerm_resource_group.my_rg.id` → the actual value to capture — reference to a resource attribute

Sensitive Outputs

```
output "db_password" {
  description = "The database password"
  value       = azurerm_sql_server.my_sql.administrator_login_password
  sensitive   = true
}
```

`sensitive = true` → hides the value in terminal output and logs — value still stored in state

⚠ NOTE: After terraform apply, terminal shows: db_password = <sensitive>

Querying Outputs

```
terraform output                    # shows all outputs
terraform output resource_group_id  # shows a specific output
```

DATA SOURCES

What is a Data Source

- Read information about EXISTING infrastructure without creating it
- resource block = "Terraform, go BUILD this"
- data block = "Terraform, go FETCH info about this existing thing"
- Data sources do NOT appear in state file — just fetched at runtime
- Use case: networking team already created VNet manually, you need to use it in your VM code

Data Source Block Syntax

```
data "azurerm_virtual_network" "existing_vnet" {
  name                = "myExistingVNet"
  resource_group_name = "networking-rg"
}
```

`data` → keyword telling Terraform to read, not create

`"azurerm_virtual_network"` → the type of resource to look up

`"existing_vnet"` → your local name for this data source inside Terraform

`name = "myExistingVNet"` → the actual name of the VNet already in Azure

`resource_group_name = "networking-rg"` → which resource group it lives in

Data Source Reference Pattern (MEMORISE THIS)

```
Format:  data.resource_type.local_name.attribute
```

```
data.azurerm_virtual_network.existing_vnet.name
data.azurerm_virtual_network.existing_vnet.id
data.azurerm_resource_group.existing_rg.id
data.azurerm_resource_group.existing_rg.location
data.azurerm_subnet.existing_subnet.address_prefixes
```

resource vs data Comparison

- **resource** — creates and manages infrastructure — appears in state
- **data** — reads existing infrastructure — does NOT appear in state

LOCALS

What are Locals

- Internal reusable values defined within your Terraform code
- Unlike variables they CANNOT be overridden from outside
- Used to avoid repeating the same values across multiple resources
- Used to compute calculated values once and reuse everywhere

Locals Block Syntax

```
locals {
  environment = "development"
  team        = "devops"
  project     = "cognizant-training"

  common_tags = {
    environment = local.environment
    team        = local.team
    project     = local.project
    managed_by  = "Terraform"
  }
}
```

`locals {}` → single block holding all local values — note plural (locals) for the block

`local.environment` → inside locals, reference another local using local. prefix (singular)

`common_tags = {}` → local that is a map — reusable set of tags applied to all resources

Local Reference Pattern (MEMORISE THIS)

```
Format:  local.name      Example:  local.common_tags
```

```
tags     = local.common_tags
location = local.default_location
name     = local.resource_prefix
```

Locals for Computed Values

```
variable "environment" {
  type    = string
  default = "dev"
}

locals {
  resource_prefix = "cognizant-${var.environment}"
  is_production   = var.environment == "prod" ? true : false
}

resource "azurerm_resource_group" "my_rg" {
  name     = "${local.resource_prefix}-rg"
  location = "East US"
}
```

`"cognizant-${var.environment}"` → string interpolation — ${} combines text with a variable value

`var.environment == "prod" ? true : false` → ternary/conditional — if environment is prod return true else false

`"${local.resource_prefix}-rg"` → if var.environment = "dev" → name becomes cognizant-dev-rg

CONDITIONAL EXPRESSIONS (Ternary)

Syntax

```
condition ? true_value : false_value
```

Example

```
resource "azurerm_linux_virtual_machine" "my_vm" {
  name  = "myVM"
  size  = var.environment == "prod" ? "Standard_D2s_v3" : "Standard_B1s"
}
```

`var.environment == "prod" ? "Standard_D2s_v3" : "Standard_B1s"` → if environment is prod → use bigger VM | else → use smaller VM

COUNT META-ARGUMENT

What is count

- Allows you to create multiple instances of the same resource
- count.index gives you the current index (starts at 0)

```
resource "azurerm_linux_virtual_machine" "my_vm" {
  count               = 3
  name                = "cognizant-vm-${count.index}"
  location            = "East US"
  resource_group_name = azurerm_resource_group.my_rg.name
  size                = "Standard_B1s"
  admin_username      = "azureuser"
}
```

`count = 3` → create 3 instances of this resource

`count.index` → current index (0, 1, 2) — gives each VM a unique name

`"cognizant-vm-${count.index}"` → creates cognizant-vm-0, cognizant-vm-1, cognizant-vm-2

Referencing Specific Instances

```
azurerm_linux_virtual_machine.my_vm[0].id   # first VM
```

```
azurerm_linux_virtual_machine.my_vm[1].id   # second VM
azurerm_linux_virtual_machine.my_vm[*].id   # all VMs as a list (splat notation)
```

BUILT-IN FUNCTIONS

- Terraform has a library of built-in functions used inside interpolations ${}

Common Functions

```
lookup(map, key)           # looks up a value in a map by key
length(list)               # returns number of items in a list or string
file("path/to/file")       # loads file contents as a string
format("Hello %s", name)   # formats a string
contains(list, value)      # returns true if list contains the value
join(",", list)            # joins list items with a separator
split(",", string)         # splits a string into a list
```

lookup() Example

```
variable "vm_sizes" {
  type = map(string)
  default = {
    dev  = "Standard_B1s"
    prod = "Standard_D2s_v3"
  }
}

resource "azurerm_linux_virtual_machine" "my_vm" {
  size = lookup(var.vm_sizes, var.environment, "Standard_B1s")
}
```

`lookup(var.vm_sizes, var.environment, "Standard_B1s")` → looks in vm_sizes map for key matching var.environment | returns "Standard_B1s" as default if not found

COMPLETE REFERENCE PATTERN SUMMARY

⚠ NOTE: Memorise ALL of these patterns — they are asked in every interview.

```
Pattern                                        Example
-------                                        -------
var.variable_name                              var.location
local.name                                     local.common_tags
resource_type.local_name.attribute             azurerm_resource_group.my_rg.name
resource_type.local_name.attribute             azurerm_virtual_network.my_vnet.id
resource_type.local_name.attribute             azurerm_subnet.my_subnet.id
resource_type.local_name.attribute
azurerm_linux_virtual_machine.my_vm.id
data.resource_type.local_name.attribute
data.azurerm_virtual_network.existing_vnet.name
data.resource_type.local_name.attribute
data.azurerm_resource_group.existing_rg.id
module.module_name.output_name                 module.networking.subnet_id
module.module_name.output_name                 module.resource_group.name
count.index                                    count.index (inside resource with
count)
self.attribute                                 self.public_ip_address (inside
provisioner)
```

```
terraform.workspace                          terraform.workspace (built-in
variable)
```

variable vs local vs output — FULL COMPARISON

```
                variable          local             output
Direction       Input→comes in    Internal          Output→goes out
Overridable?    Yes               No                N/A
Who sets it?    Person running TF Code author       Terraform after apply
Reference       var.              local.            N/A — displayed to terminal
Use case        Dynamic inputs    Avoid repetition  Display results after apply
```

PHASE 4 — THE WORKFLOW

terraform init

- Initialises your working directory
- Downloads provider plugins from Terraform Registry
- Sets up remote backend if configured
- Downloads modules if used
- Creates .terraform folder (holds downloaded plugins)
- Creates .terraform.lock.hcl (locks exact provider version for team consistency)

```
terraform init
```

- When to run:
  - First time starting a project
  - When you add a new provider
  - When you add a new module
  - When you change backend configuration

terraform plan

- Preview of changes — NOTHING gets created or destroyed
- Compares your .tf code with the current state file
- Shows what will be added, changed or destroyed

```
terraform plan
```

Plan Output Symbols

```
+     resource will be CREATED
-     resource will be DESTROYED
~     resource will be UPDATED (modified in place)
-/+   resource will be DESTROYED and RECREATED
      (happens when a change cannot be done in place e.g. changing VM image)
(known after apply)  value will be assigned by Azure after creation
```

Save Plan to File (Best Practice)

```
terraform plan -out=tfplan   # save the plan
terraform apply tfplan       # apply exactly what was reviewed
```

terraform apply

- Actually creates, updates or destroys resources in Azure
- Calls Azure APIs based on your code and state
- Shows the plan one more time then requires manual confirmation (type yes)

```
terraform apply               # apply with confirmation prompt
terraform apply tfplan        # apply a saved plan
terraform apply -auto-approve # skip confirmation (for CI/CD pipelines)
```

terraform destroy

- Destroys ALL infrastructure managed by Terraform
- Reads state file to know what to delete
- Prompts for confirmation before destroying

```
terraform destroy               # destroy with confirmation prompt
terraform destroy -auto-approve # skip confirmation
```

⚠ **NOTE: Always double-check before running terraform destroy in production!**

terraform fmt and terraform validate

```
terraform fmt       # automatically formats .tf files to standard style
terraform validate  # checks syntax errors without connecting to Azure
```

- Run BOTH before every commit to version control

terraform console

- Interactive REPL to test functions and expressions before using in code

```
terraform console
> length(["a","b","c"])
3
> lookup({dev="small"}, "dev")
"small"
```

COMPLETE WORKFLOW

```
1. Write your .tf files
2. terraform init      → download providers, setup backend
3. terraform validate  → check for syntax errors
4. terraform fmt       → format code
5. terraform plan      → preview what will change
6. terraform apply     → make it happen on Azure
7. terraform output    → check output values
8. terraform destroy   → clean up when done
```

COMMANDS SUMMARY TABLE

```
Command                Touches Azure?   Purpose
-------------------------------------------------------------------
terraform init         No               Setup — download providers and modules
terraform plan         No (read only)   Preview changes
terraform apply        Yes              Make changes
terraform destroy      Yes              Delete everything
terraform fmt          No               Format code
terraform validate     No               Check syntax
terraform output       No               Show output values
terraform show         No               Show state file contents
```

```
terraform state list  No               List tracked resources
terraform console     No               Interactive function testing
terraform import      Yes (read only)  Import existing resource into state
```

PHASE 5 — REAL WORLD STRUCTURE

MODULES

What is a Module

- A folder containing Terraform files
- Organises infrastructure into logical, reusable chunks
- Every directory with .tf files is technically a module
- Root module — your main project folder, entry point, coordinates everything
- Child module — a reusable sub-folder called by root, does one specific job

Module File Structure (every module follows this)

```
main.tf       → resource definitions
variables.tf  → inputs this module accepts
outputs.tf    → values this module exposes to others
```

Module Sources

```
source = "./modules/networking"          # local path
source = "hashicorp/consul/aws"          # Terraform Registry
source = "github.com/hashicorp/example"  # GitHub
source = "s3::https://..."              # S3 bucket
```

PROJECT FOLDER STRUCTURE

```
terraform-azure-project/
│
├── main.tf               → root module — calls all child modules
├── variables.tf          → input variables for root module
├── outputs.tf            → outputs from root module
├── terraform.tfvars      → actual values for variables
│
└── modules/
    ├── resource_group/
    │   ├── main.tf       → creates the resource group
    │   ├── variables.tf  → inputs this module needs
    │   └── outputs.tf    → values this module exposes
    │
    ├── networking/
    │   ├── main.tf       → creates VNet and Subnet
    │   ├── variables.tf
    │   └── outputs.tf
    │
    └── virtual_machine/
        ├── main.tf       → creates the VM
        ├── variables.tf
        └── outputs.tf
```

MODULE 1 — Resource Group

modules/resource_group/main.tf

```
resource "azurerm_resource_group" "rg" {
  name     = var.name
  location = var.location
  tags     = var.tags
}
```

modules/resource_group/variables.tf

```
variable "name" {
  description = "Name of the resource group"
  type        = string
}

variable "location" {
  description = "Azure region"
  type        = string
}

variable "tags" {
  description = "Tags to apply"
  type        = map(string)
  default     = {}
}
```

modules/resource_group/outputs.tf

```
output "name" {
  value = azurerm_resource_group.rg.name
}

output "location" {
  value = azurerm_resource_group.rg.location
}
```

MODULE 2 — Networking

modules/networking/main.tf

```
resource "azurerm_virtual_network" "vnet" {
  name                = var.vnet_name
  location            = var.location
  resource_group_name = var.resource_group_name
  address_space       = var.address_space
  tags                = var.tags
}

resource "azurerm_subnet" "subnet" {
  name                 = var.subnet_name
  resource_group_name  = var.resource_group_name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = var.subnet_prefixes
}
```

modules/networking/outputs.tf

```
output "vnet_id" {
  value = azurerm_virtual_network.vnet.id
}

output "subnet_id" {
  value = azurerm_subnet.subnet.id
}
```

MODULE 3 — Virtual Machine

modules/virtual_machine/main.tf

```
resource "azurerm_network_interface" "nic" {
  name                = "${var.vm_name}-nic"
  location            = var.location
  resource_group_name = var.resource_group_name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = var.subnet_id
    private_ip_address_allocation = "Dynamic"
  }
}

resource "azurerm_linux_virtual_machine" "vm" {
  name                  = var.vm_name
  location              = var.location
  resource_group_name   = var.resource_group_name
  size                  = var.vm_size
  admin_username        = var.admin_username
  network_interface_ids = [azurerm_network_interface.nic.id]

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "UbuntuServer"
    sku       = "18.04-LTS"
    version   = "latest"
  }

  tags = var.tags
}
```

modules/virtual_machine/outputs.tf

```
output "vm_id" {
  value = azurerm_linux_virtual_machine.vm.id
}
```

```
output "private_ip" {
  value = azurerm_network_interface.nic.private_ip_address
}
```

ROOT MODULE — main.tf (Gluing Everything Together)

```
locals {
  common_tags = {
    environment = var.environment
    managed_by  = "Terraform"
    project     = "cognizant-training"
  }
}

module "resource_group" {
  source   = "./modules/resource_group"
  name     = "cognizant-${var.environment}-rg"
  location = var.location
  tags     = local.common_tags
}

module "networking" {
  source              = "./modules/networking"
  vnet_name           = "cognizant-${var.environment}-vnet"
  subnet_name         = "cognizant-${var.environment}-subnet"
  location            = module.resource_group.location
  resource_group_name = module.resource_group.name
  address_space       = ["10.0.0.0/16"]
  subnet_prefixes     = ["10.0.1.0/24"]
  tags                = local.common_tags
}

module "virtual_machine" {
  source              = "./modules/virtual_machine"
  vm_name             = "cognizant-${var.environment}-vm"
  location            = module.resource_group.location
  resource_group_name = module.resource_group.name
  subnet_id           = module.networking.subnet_id
  vm_size             = var.vm_size
  tags                = local.common_tags
}
```

`source = "./modules/resource_group"` → where to find the module — local path

`name = "cognizant-${var.environment}-rg"` → passing a dynamically built name into module using string interpolation

`module.resource_group.location` → networking module gets its location from the resource group module output

`module.resource_group.name` → networking module gets RG name from resource group module output

`module.networking.subnet_id` → VM module gets its subnet ID from the networking module output

Module Output Reference Pattern (MEMORISE THIS)

Format: `module.module_name.output_name`

```
module.resource_group.name
module.resource_group.location
```

```
module.networking.subnet_id
module.networking.vnet_id
module.virtual_machine.vm_id
module.virtual_machine.private_ip
```

HOW MODULES CONNECT

```
Root Module (main.tf)
      │
      ├──▶ Resource Group Module
      │          └── outputs: name, location
      │                  │
      ├──▶ Networking Module ◀── receives: location, resource_group_name
      │          └── outputs: vnet_id, subnet_id
      │                  │
      └──▶ VM Module ◀──────── receives: location, resource_group_name,
subnet_id
```

⚠ NOTE: One module's output becomes another module's input. This is the core concept of modular Terraform.

TERRAFORM IMPORT

- Used to bring existing infrastructure (created outside Terraform) under Terraform management
- Terraform creates an entry in the state file for that resource
- You must first define the resource block in your .tf file manually

Steps to Import a Resource

- Step 1 — Create the resource block in your .tf file

```
resource "azurerm_resource_group" "existing" {
  name     = "myExistingRG"
  location = "East US"
}
```

- Step 2 — Run terraform init
- Step 3 — Run terraform import

```
terraform import azurerm_resource_group.existing
/subscriptions/xxx/resourceGroups/myExistingRG

Format:  terraform import resource_type.local_name <azure_resource_id>
```

- Step 4 — Verify with terraform show

PHASE 6 — INTERVIEW FAVOURITES

STATE LOCKING AND REMOTE BACKENDS

Problem Without Locking

- Two engineers run terraform apply simultaneously
- Both read the same state file at the same time
- Both write back to state simultaneously

- State file gets corrupted — Terraform loses track of what exists

Azure Remote Backend + Built-in Locking

```
terraform {
  backend "azurerm" {
    resource_group_name  = "tfstate-rg"
    storage_account_name = "cognizanttfstate"
    container_name       = "tfstate"
    key                  = "prod/terraform.tfstate"
  }
}
```

⚠ NOTE: Azure Blob Storage has state locking built in via blob leasing. AWS needs a separate DynamoDB table. Azure is automatic.

Emergency Unlock

```
terraform force-unlock <LOCK_ID>   # only use if 100% sure no apply is running
```

LIFECYCLE RULES

prevent_destroy

- Terraform refuses to destroy this resource even if terraform destroy is run
- Use case: production databases, critical resource groups

```
resource "azurerm_resource_group" "production_rg" {
  name     = "production-rg"
  location = "East US"

  lifecycle {
    prevent_destroy = true
  }
}
```

`prevent_destroy = true` → Terraform throws an error instead of deleting — protects critical resources

ignore_changes

- Terraform ignores any changes to listed attributes
- Use case: when another team or Azure policy manages certain attributes externally

```
resource "azurerm_linux_virtual_machine" "vm" {
  name                = "cognizant-vm"
  location            = "East US"
  resource_group_name = "cognizant-rg"
  size                = "Standard_B1s"
  admin_username      = "azureuser"

  lifecycle {
    ignore_changes = [
      tags,
      size
    ]
  }
}
```

`ignore_changes = [tags, size]` → Terraform ignores any changes to tags and size even if they change in code or in Azure

create_before_destroy

- Default Terraform behaviour → destroy first, then create → causes downtime
- create_before_destroy → create new one first, then destroy old → zero downtime
- Use case: production VMs, load balancers, anything where downtime is unacceptable

```
resource "azurerm_linux_virtual_machine" "vm" {
  name                = "cognizant-vm"
  location            = "East US"
  resource_group_name = "cognizant-rg"
  size                = "Standard_B1s"
  admin_username      = "azureuser"

  lifecycle {
    create_before_destroy = true
  }
}
```

Lifecycle Rules Summary

```
prevent_destroy        → block accidental deletion of critical resources
ignore_changes         → ignore specific attribute changes
create_before_destroy  → zero downtime replacements
```

PROVISIONERS

What are Provisioners

- Run scripts or commands on a resource AFTER Terraform creates it
- Used for bootstrapping — installing software, configuring settings
- Defined inside a resource block using a provisioner block

⚠ NOTE: Provisioners are a LAST RESORT. Use native Azure VM extensions or cloud-init when possible.

remote-exec Provisioner

- Runs commands on the remote resource via SSH

```
resource "azurerm_linux_virtual_machine" "vm" {
  name                = "cognizant-vm"
  location            = "East US"
  resource_group_name = "cognizant-rg"
  size                = "Standard_B1s"
  admin_username      = "azureuser"

  provisioner "remote-exec" {
    inline = [
      "sudo apt-get update",
      "sudo apt-get install -y nginx",
      "sudo systemctl start nginx"
    ]
  }
```

```
  connection {
    type        = "ssh"
    user        = "azureuser"
    private_key = file("~/.ssh/id_rsa")
    host        = self.public_ip_address
  }
}
```

`provisioner "remote-exec"` → after VM is created, SSH into it and run these commands

`inline = [...]` → list of shell commands to run in order on the remote VM

`connection {}` → tells Terraform how to connect to the VM

`host = self.public_ip_address` → self refers to the resource this provisioner belongs to

`private_key = file("~/.ssh/id_rsa")` → file() function loads the private key from disk

local-exec Provisioner

- Runs commands on YOUR local machine where Terraform is running

```
resource "azurerm_resource_group" "rg" {
  name     = "cognizant-rg"
  location = "East US"

  provisioner "local-exec" {
    command = "echo Resource Group ${self.name} created in ${self.location} >>
deployment.log"
  }
}
```

`provisioner "local-exec"` → after resource is created, run this command on YOUR local machine

`self.name` → reference the resource's own name attribute

`>> deployment.log` → append output to a local log file

file Provisioner

- Copies files from local machine to the remote resource

```
provisioner "file" {
  source      = "configs/nginx.conf"
  destination = "/etc/nginx/nginx.conf"
}
```

`source` → where to find the file on your local machine

`destination` → where to put the file on the remote resource

Connection Scoping

- Connection inside resource block → all provisioners in that resource use it
- Connection inside provisioner block → only that specific provisioner uses it
- Use case for multiple connections: connect as root first to set up users, then connect as new user to install software

WORKSPACES

What Workspaces Solve

- Manage multiple environments (dev, staging, prod) using the same Terraform code
- Each workspace has its own separate state file
- Without workspaces you would need duplicate code folders for each environment

Workspace Commands

```
terraform workspace list           # list all workspaces
terraform workspace new dev        # create a new workspace named dev
terraform workspace select prod    # switch to prod workspace
terraform workspace show           # display current workspace name
```

Using Workspace in Code

```
locals {
  environment = terraform.workspace
}

resource "azurerm_resource_group" "rg" {
  name     = "cognizant-${local.environment}-rg"
  location = "East US"
}
```

`terraform.workspace` → built-in variable that returns the current workspace name

- In dev workspace → creates cognizant-dev-rg
- In prod workspace → creates cognizant-prod-rg
- Same code, separate state files per workspace

BEST PRACTICES

- 1. Always use remote state for teams

```
backend "azurerm" {
  storage_account_name = "tfstate"
  container_name       = "tfstate"
  key                  = "terraform.tfstate"
}
```

- 2. Always run terraform plan before apply

```
terraform plan -out=tfplan
terraform apply tfplan
```

- 3. Pin provider and Terraform versions

```
terraform {
  required_version = ">= 1.0.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}
```

- 4. Never hardcode secrets

```
# WRONG
provider "azurerm" {
  client_secret = "mysecretpassword123"
}

# RIGHT
provider "azurerm" {
  client_secret = var.client_secret
}
```

- 5. Run terraform fmt and terraform validate always

```
terraform fmt && terraform validate
```

- 6. Modularize your code — never put everything in one file for production
- 7. Use meaningful variable names with descriptions always
- 8. Use variable validation to enforce constraints on inputs
- 9. Store all Terraform code in Git — NEVER commit these files:

```
.terraform/                → downloaded provider plugins
terraform.tfstate          → state file (contains sensitive data)
terraform.tfstate.backup   → state backup
*.tfvars                   → may contain secrets
```

- 10. Enable versioning and encryption on remote state storage
  - Versioning → allows recovery from a previous state
  - Encryption → protects sensitive data in state at rest

⚠ NOTE: When interviewer asks "what are Terraform best practices?" — name these 10 points confidently.